Ngrams and Language Modeling

Md Shad Akhtar
shad.akhtar@iiitd.ac.in

INDRAPRASTHA INSTITUTE of INFORMATION TECHNOLOGY DELHI

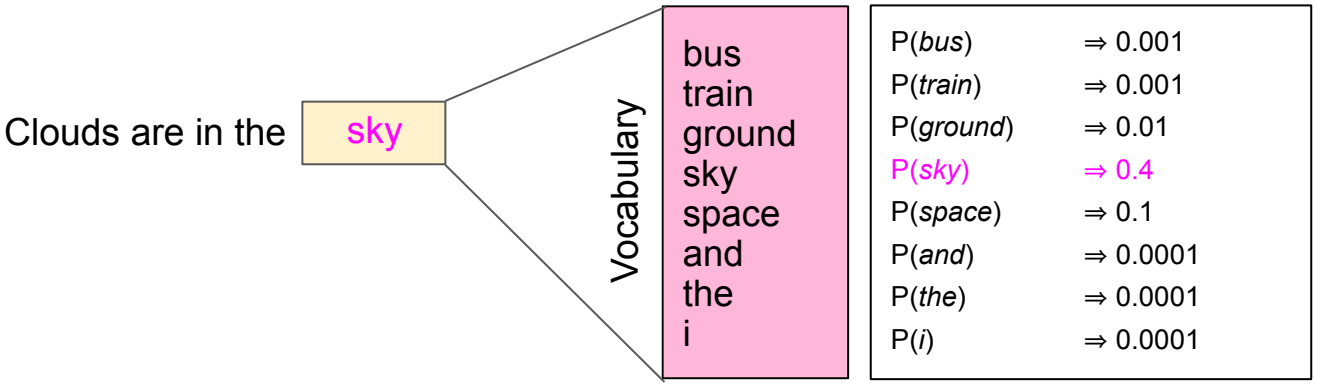
N-grams

- Taking N tokens at a time.
  - “Morphological parsing is the process of finding the constituent morphemes.”
    - 1-grams: “*morphological*”, “*parsing*”, “*is*”, “*the*”, “*process*”, “*of*”, “*finding*”, “*the*”, “*constituent*”, “*morphemes*”, “.”
      - Tokens: 11
      - Types (unique tokens): 10
    - 2-grams: “*Morphological parsing*”, “*parsing is*”, “*is the*”, “*the process*”, “*process of*”, “*of finding*”, “*finding the*”, “*the constituent*”, “*constituent morphemes*”, “*morphemes .*”
    - 3-grams: “*Morphological parsing is*”, “*parsing is the*”,....
    - …
    - n-grams: “$w_i w_{i+1} \cdots w_{i+n-1}$”, …..

Word Prediction

- Given a sequence of words in an incomplete sentence, what is the most probable next word
  - In today's world, everyone is obsessed with deep

learning
models
systems
space
well
…



$P(w_i \mid w_1, w_2, \ldots, w_{i-1})$

P(bus | *Clouds are in the*)
P(train | *Clouds are in the*)
P(ground | *Clouds are in the*)
P(sky | *Clouds are in the*)
P(space | *Clouds are in the*)
P(and | *Clouds are in the*)
P(the | *Clouds are in the*)
P(i | *Clouds are in the*)

Language Model

Most probable sequence $P(W) = P(w_1, w_2, \cdots, w_n)$

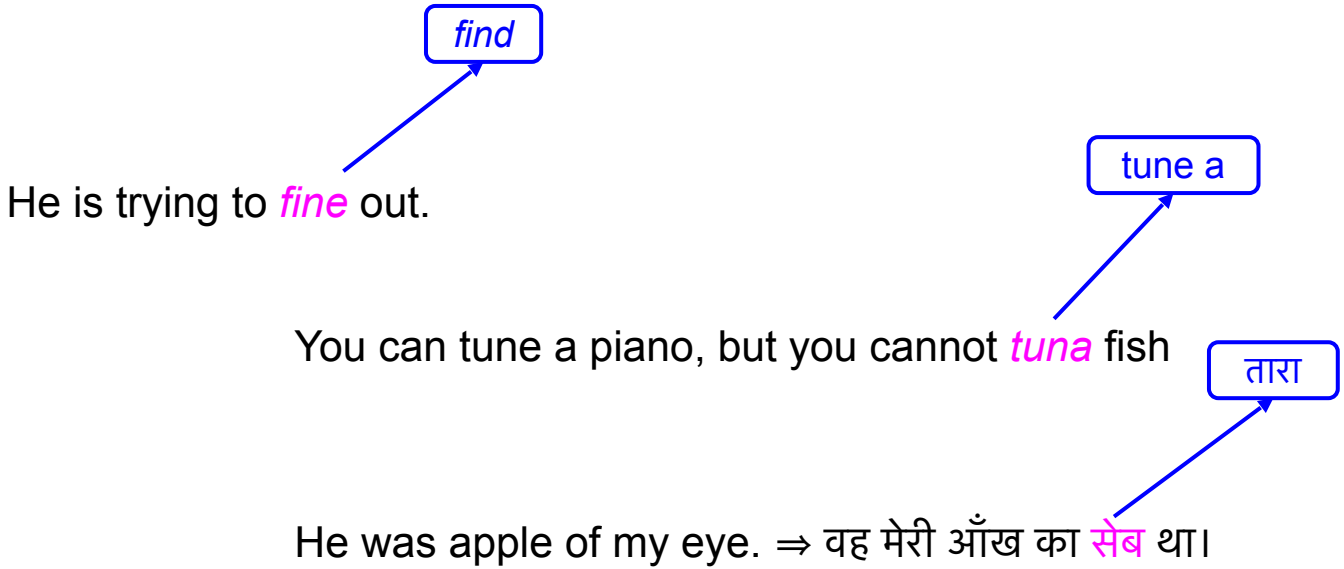
P(*Clouds are in the sky*) > P(*Clouds are in the bus*)

Most probable word in the sequence $P(w_n \mid w_1, w_2, \cdots, w_{n-1})$

P(*sky* | *Clouds are in the*) > P(*bus* | *Clouds are in the*)

Applications

- Spelling correction
- Speech recognition
- Any generation task
  - MT



Estimating probabilities

$$P(W) = P(w_1, w_2, \cdots, w_n)$$

Apply chain-rule (Joint probability → Sequence of conditional probabilities)

$$P(W) = P(w_1) \,.\, P(w_1 \,|\, w_2) \,.\, P(w_3 \,|\, w_1 w_2) \,.\, P(w_4 \,|\, w_1 w_2 w_3) \cdots P(w_n \,|\, w_1 w_2 w_3 \cdots w_{n-1})$$

$$= \Pi_1^n \quad P(w_i \,|\, w_1 w_2 w_3 \cdots w_{i-1})$$

P(clouds are in the sky) = P(clouds, are, in, the, sky)

= P(clouds) P(are | clouds) P(in | clouds are) P(the | clouds are in) P(sky | clouds are in the)

Estimating probabilities

- We can utilize the occurrences of words for estimating probabilities

$$P(w_i \mid w_1 w_2 w_3 \cdots w_{i-1}) = \frac{count(w_1 w_2 w_3 \cdots w_{i-1} w_i)}{count(w_1 w_2 w_3 \cdots w_{i-1})}$$

$$P(sky \mid \text{clouds are in the}) = \frac{count(\text{clouds are in the } sky)}{count(\text{clouds are in the})}$$

$$P(\text{clouds are in the sky}) = \left[\frac{C(clouds)}{|V|}\right]\left[\frac{C(\text{clouds } are)}{C(\text{clouds})}\right]\left[\frac{C(\text{clouds are } in)}{C(\text{clouds are})}\right]\left[\frac{C(\text{clouds are in } the)}{C(\text{clouds are in})}\right]\left[\frac{C(\text{clouds are in the } sky)}{C(\text{clouds are in the})}\right]$$

Corpora

- How do we get this counts?
  - From a corpora
    - A (large) set of sentences
    - E.g.,
      - Brown corpus (1M words)
      - Switchboard corpus (2.4M words)
      - Google News corpus (100B words)

N-gram models

- Though the idea of estimating probabilities is straightforward and simple, but does not work well in practice
  - Any language can have too many possible sentences
  - We can’t see everything in our corpus.

- Solution: Approximate it.
- Markov Assumption
  - Probability of a word can be estimated without looking too far in the past

N-gram models

$$P(sky \mid \text{clouds are in the}) \approx P(sky \mid \text{the}) = \frac{count(\text{the } sky)}{count(\text{the})}$$ (Bigram model)

$$P(sky \mid \text{clouds are in the}) \approx P(sky \mid \text{in the}) = \frac{count(\text{in the } sky)}{count(\text{in the})}$$ (Trigram model)

$$P(w_i \mid w_1 w_2 \cdots w_{i-1}) \approx P(w_i \mid w_{i-n+1} \cdots w_{i-1}) = \frac{count(w_{i-n+1} \cdots w_{i-1} w_i)}{count(w_{i-n+1} \cdots w_{i-1})}$$ (n-gram model)

N-gram models - examples

- WSJ corpus
  - Bigram
    - Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her
  - Trigram
    - They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions.

N-gram models

- As we extend to higher n-grams, we get more coherent text.

- Few issues:
  - Higher n-gram models are inefficient
  - Long-term dependency

- In practice, 3-4-5 grams works reasonably well.

Bigram Model - Corpus

- Berkeley Restaurant Corpus:
  - can you tell me about any good cantonese restaurants close by
  - mid priced thai food is what i'm looking for
  - tell me about chez panisse
  - can you give me a listing of the kinds of food that are available
  - i'm looking for a good place to eat breakfast
  - when is caffe venezia open during the day
  - …
  - ...

~9K sentences

Bigram Model - Co-occurrence Matrix

Bigram count

	I	want	to	eat	Chinese	food	lunch
I	8	1087	0	13	0	0	0
want	3	0	786	0	6	8	6
to	3	0	10	860	3	0	12
eat	0	0	2	0	19	2	52
Chinese	2	0	0	0	0	120	1
food	19	0	17	0	0	0	0
lunch	4	0	0	0	0	1	0

. . . .

Unigram count

I	3437
want	1215
to	3256
eat	938
Chinese	213
food	1506
lunch	459

Bigram probability

	I	want	to	eat	Chinese	food	lunch
I	.0023	.32	0	.0038	0	0	0
want	.0025	0	.65	0	.0049	.0066	.0049
to	.00092	0	.0031	.26	.00092	0	.0037
eat	0	0	.0021	0	.020	.0021	.055
Chinese	.0094	0	0	0	0	.56	.0047
food	.013	0	.011	0	0	0	0
lunch	.0087	0	0	0	0	.0022	0

P(I want Chinese food)

*= P(I) * P(want | I) * P(Chinese | wants) * P(food | Chinese)*

*= 0.31 * 0.32 * 0.0049 * 0.56*
= 0.00027

Issue with estimating probability

- As sequence length increases, the overall probability gets smaller
  - Risk of numeric underflow

- Compute probability in log space

$$\log \left[\Pi_{i=1}^{n} \quad P(w_i \,|\, w_{i-1}))\right] = \sum_{i=1}^{n} \quad \log \left[P(w_i \,|\, w_{i-1}))\right]$$

Language Model Evaluation

- Given two language models A and B (possibly on the same corpus), which one is better?
- Evaluation
  - Human evaluation → Check how likely the generated sentence is
    - Too costly to evaluate all the generated sentences
  - Extrinsic evaluation → Utilize the generate sentence for building another system (e.g., MT)
    - Not an idealistic solution. Some external system may take hours/days/weeks to train
  - Intrinsic evaluation → Compute some evaluation metric
    - Give better scores to high-probable sentences.
    - Perplexity
      - Not very accurate measure and depends heavily on the corpus the LM trained on.
      - But provide good approximation

Perplexity

- Perplexity is the inverse probability of the target sentence, normalized by the number of words:

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

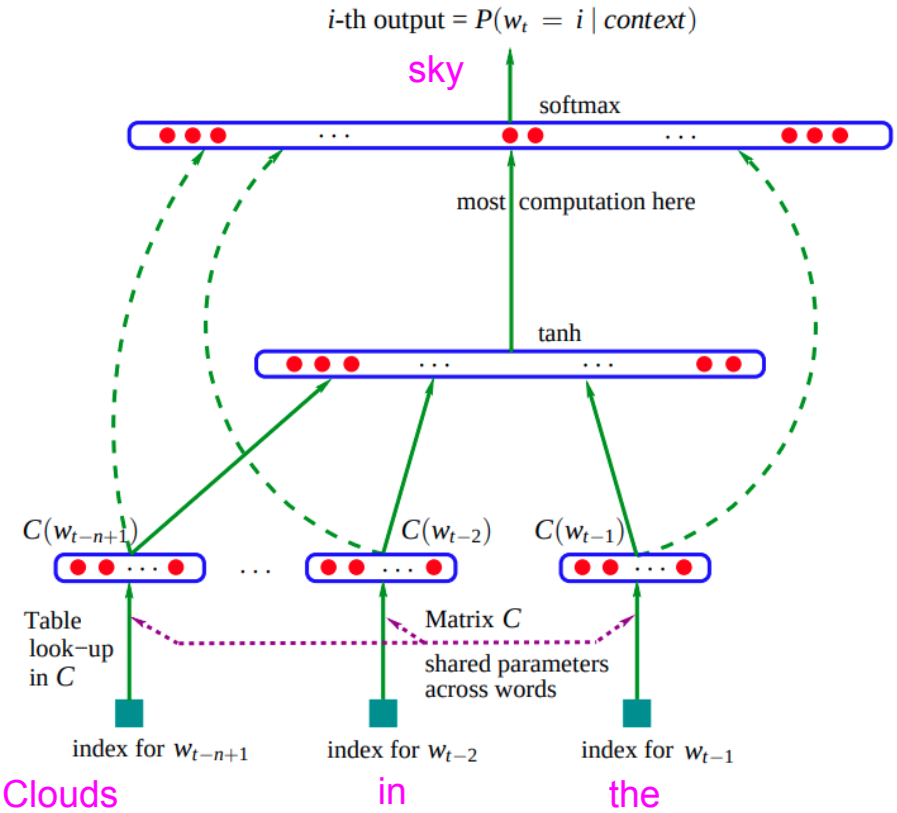
Lower perplexity is better!

For Bigram model

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1})}}$$

https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf

Neural Language Modeling: MLP - [Bengio et al., 2003]



- Input layer
  - n previous words
- Projection layer (no non-linearity)
  - Projection matrix C of size |V| x m
- Hidden layer (tanh)
- Output layer
- Skip-connection (optional)

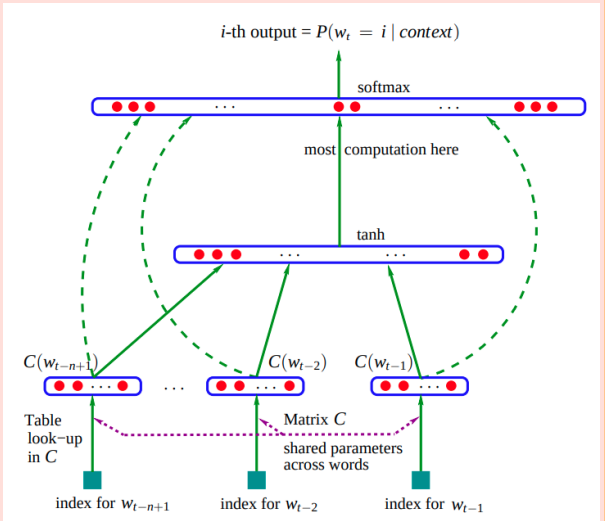
$$y = \text{softmax}(b + Wx + U \tanh(Hx + d))$$

$$x = [C(w_{t-1}), C(w_{t-2}), ..., C(w_{t-n+1})]$$

https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf

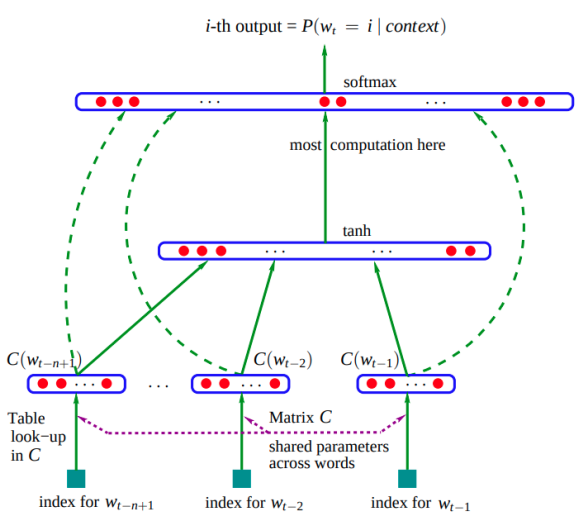
Issue with MLP-based Neural LM

sky



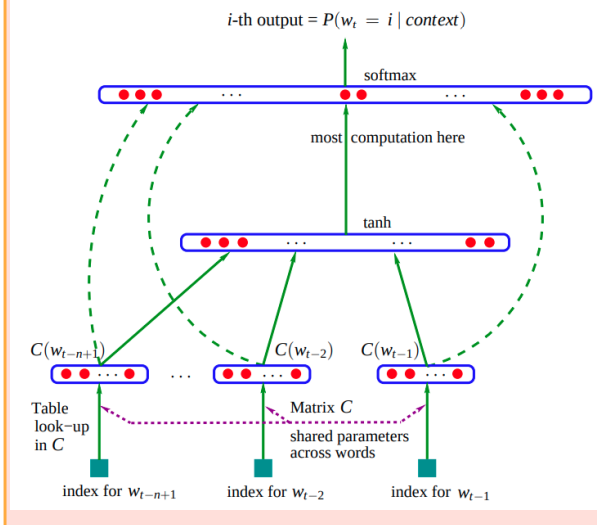
in Clouds the

sky



the in Clouds

sky

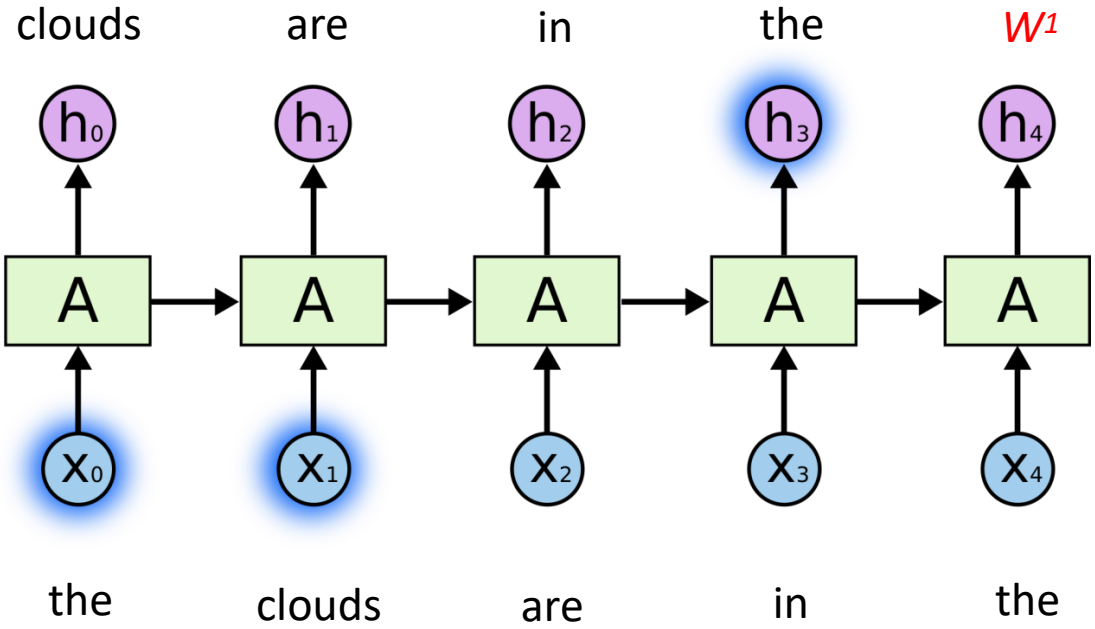


Clouds in the

All these are essentially the same.

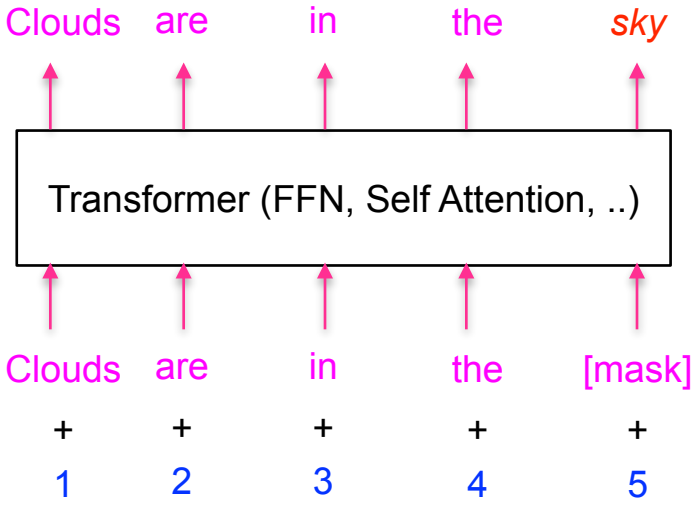
Neural Language Modeling: RNN

- "the clouds are in the *sky*"



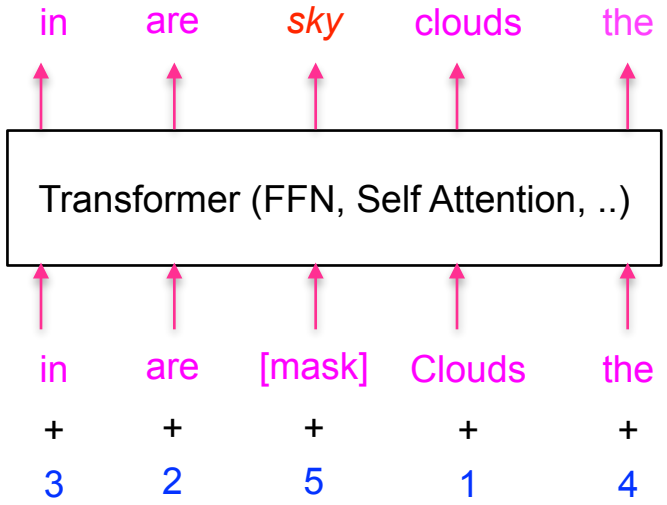
Neural Language Modeling: Transformer-based

- "the clouds are in the *sky*"



Positions*

Actual positional encoding are different



Computationally, both are same.

Out-of-vocabulary (OOVs) words and Zero probabilities

Zero probability

- What if, we want to estimate the probability of
  - A word that never occurred in the training set
  - A bigram that never co-occurred in the training set

Train set:

- *I want Chinese food*
- *I want Italian food*
- *I want Indian food*
- *...*

Test set:

- *I want British food*

P(British | want) = 0

- Their probability will be zero, hence, the perplexity of the sentence will be infinity.

Add-1 (or Laplace) Smoothing

	I	want	to	eat	Chinese	food	lunch
I	8	1087	0	13	0	0	0
want	3	0	786	0	6	8	6
to	3	0	10	860	3	0	12
eat	0	0	2	0	19	2	52
Chinese	2	0	0	0	0	120	1
food	19	0	17	0	0	0	0
lunch	4	0	0	0	0	1	0

	I	want	to	eat	Chinese	food	lunch
I	.0023	.32	0	.0038	0	0	0
want	.0025	0	.65	0	.0049	.0066	.0049
to	.00092	0	.0031	.26	.00092	0	.0037
eat	0	0	.0021	0	.020	.0021	.055
Chinese	.0094	0	0	0	0	.56	.0047
food	.013	0	.011	0	0	0	0
lunch	.0087	0	0	0	0	.0022	0

Steal a small fraction of probability from the non-zero terms and redistribute it over the zero occurring terms

Assume every word/bigram has occurred at least once in the corpus.

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}w_i) + 1}{count(w_{i-1}) + V}$$

If the co-occurrence matrix is too sparse, Add-1 smoothing does not work well.

	I	want	to	eat	Chinese	food	lunch
I	9	1088	1	14	1	1	1
want	4	1	787	1	7	9	7
to	4	1	11	861	4	1	13
eat	1	1	3	1	20	3	53
Chinese	3	1	1	1	1	121	2
food	20	1	18	1	1	1	1
lunch	5	1	1	1	1	2	1

	I	want	to	eat	Chinese	food	lunch
I	.0018	.22	.00020	.0028	.00020	.00020	.00020
want	.0014	.00035	.28	.00035	.0025	.0032	.0025
to	.00082	.00021	.0023	.18	.00082	.00021	.0027
eat	.00039	.00039	.0012	.00039	.0078	.0012	.021
Chinese	.0016	.00055	.00055	.00055	.00055	.066	.0011
food	.0064	.00032	.0058	.00032	.00032	.00032	.00032
lunch	.0024	.00048	.00048	.00048	.00048	.00096	.00048

V = 1616 in Berkeley Restaurant Corpus

Add-1 (or Laplace) Smoothing

- V = [a, b, w]
- Corpus
  - “a b b a a w a b w b”
  - “w a a a b a”
- What should be the next word in the sequence “.....w ___”?
- Bi-gram Probability

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}w_i)}{count(w_{i-1})}$$

P_a(a | w) = count(w a) / count (w) = 2/3 = 0.66
P_b(b | w) = count(w b) / count (w) = 1/3 = 0.33
P_w(w | w) = count(w w) / count (w) = 0/3 = 0

$$\sum P = 1$$

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}w_i) + 1}{count(w_{i-1})}$$

P_a(a | w) = count(w a) / count (w) = 2+1/3 = 1.0
P_b(b | w) = count(w b) / count (w) = 1+1/3 = 0.66
P_w(w | w) = count(w w) / count (w) = 0+1/3 = 0.33

$$\sum P = 2$$

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}w_i) + 1}{count(w_{i-1}) + V}$$

P_a(a | w) = count(w a) / count (w) = 3/6 = 0.5
P_b(b | w) = count(w b) / count (w) = 2/6 = ~0.33
P_w(w | w) = count(w w) / count (w) = 1/6 = ~0.17

$$\sum P = 1$$

How to know a smoothing is bad or good?

- Unigram probability: $P(x) = \frac{C_x}{N}$
- Reconstructed counts: $C_x^* = P(x) * N,$ where $C_x^* \approx C_x$
- Bigram probability: $P(x \mid y) = \frac{C_{y,x}}{C_y}$
- Reconstructed counts: $C_{y,x}^* = P(x \mid y) * C_y,$ where $C_{y,x}^* \approx C_{y,x}$

- Bigram probability with L-smoothing function *f (.)*: $P(x \mid y) = f\left(\frac{C_{y,x}}{C_y}\right) = \frac{C_{y,x} + 1}{C_y + V}$
- Reconstructed counts: $C_{y,x}^* = P(x \mid y) * C_y,$ where $C_{y,x}^* \approx C_{y,x}$

Any smoothing function *f (.)* is good if it approximates *C** as accurately as possible.

Reconstructed counts for the previous example

$$C^*(w_i \mid w_{i-1}) = \frac{(C_{w_{i-1},w_i} + 1)C_{w_{i-1}}}{C_{w_{i-1}} + V}$$

	I	want	to	eat	Chinese	food	lunch
I	6	740	.68	10	.68	.68	.68
want	2	.42	331	.42	3	4	3
to	3	.69	8	594	3	.69	9
eat	.37	.37	1	.37	7.4	1	20
Chinese	.36	.12	.12	.12	.12	15	.24
food	10	.48	9	.48	.48	.48	.48
lunch	1.1	.22	.22	.22	.22	.44	.22

Reconstructed count

	I	want	to	eat	Chinese	food	lunch
I	8	1087	0	13	0	0	0
want	3	0	786	0	6	8	6
to	3	0	10	860	3	0	12
eat	0	0	2	0	19	2	52
Chinese	2	0	0	0	0	120	1
food	19	0	17	0	0	0	0
lunch	4	0	0	0	0	1	0

Original count

Types of Smoothing

- Additive smoothing
  - Add-1 or Laplace
  - Add-*k*
- Backoff
- Interpolation
- Witten-Bell
- Good-Turing
- Absolute discounting
- Kneser-Ney

Add-*k*

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}w_i) + k}{count(w_{i-1}) + kV}$$

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}w_i) + \frac{m}{V}}{count(w_{i-1}) + m} \qquad \text{let, } m = kV$$

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}w_i) + mP(w_i)}{count(w_{i-1}) + m}$$

Backoff and Interpolation

- Backoff
  - Start with n-gram,
  - If insufficient observations, check $(n-1)$gram
  - If insufficient observations, check $(n-2)$gram
  - ...

- Interpolation
  - Try mixture of (multiple) n-gram models

$$P(w_i \mid w_{i-2}w_{i-1}) = \alpha_1 P(w_i \mid w_{i-2}w_{i-1}) + \alpha_2 P(w_i \mid w_{i-1}) + \alpha_3 P(w_i), \qquad \text{subjected to } \sum \alpha_i = 1$$

Advanced Smoothing

- Witten Bell
- Good Turing
- Kneser-Ney

- Intuition
  - To distribute some probability mass (m_u) to all unseen (ngram-with-zero-count) samples, take it (m_u) from the probability mass (m_s) of samples seen just once.

Witten-Bell Smoothing

- Assume, we have seen *N* words and these *N* words belong to *T* word types
- What is *P(*words that have been seen for the very first time)?
  - It is $\frac{T}{N}$
    - When we see a word for the first time → we are seeing it's type for the first time as well
- What is the *probability mass* of all unseen words?
  - Recall our intuition: take this mass from the words that are seen just once.
    - It is $\frac{T}{N+T}$
- Now, this probability mass needs to be distributed to all n-grams with zero counts.
  - Let ***z*** be number of word type with zero counts, *P** is the new probability of the word which had zero-probability earlier

$$P^* = \frac{T}{(N+T) * z}$$

X

Witten-Bell Smoothing

- An example: [N = 100, T = 10, z=5]
  - The *probability mass* of words seen for the first time: $\frac{T}{N}$ 10/100 = 0.1
  - The *probability mass* of all unseen words: $\frac{T}{N+T}$ 10/110 = 0.09
  - So, the *probability mass* of words seen for the first time after discount: 0.1 - 0.09 = 0.01
  - The *probability mass* of all seen words: $\frac{C_{w_{i-1}w_i}}{C_{w_{i-1}}+T} = \frac{C_{w_{i-1}w_i}}{N+T}$ 99/110 = 0.9

$$c_i^* = \begin{cases} \frac{T}{Z}\frac{N}{N+T}, & \text{if } c_i = 0 \\ c_i\frac{N}{N+T}, & \text{if } c_i > 0 \end{cases}$$

Discounted probability

Good-Turing Smoothing

- Intuition is similar
- Instead of estimating the *probability mass* of word types for the first time and distribute it to the unseen words, it estimates the *probability mass* for the words seen once.

- Let's V= [a,b,c,d,e,f,g,h] and our observation from some corpus is
  - a:10, b:5, c:5, d:2, e:1, f:1, g:1 ⇒ N = 25
  - P(e) = 1/25 = 0.04
  - P(h) = ??

- The *probability mass* of words seen once?
  - P^* = 3/25

Once we take 3/25 for unseen words, what will happen to

P(e) = ?
P(f) = ?
P(g) = ?

It will be distributed among unseen words

Good-Turing Smoothing

- Let's V= [a,b,c,d,e,f,g,h] and our observation from some corpus is
  - a:10, b:5, c:5, d:2, e:1, f:1, g:1 ⇒ N = 25
  - P(e) = 1/25 = 0.04
  - P(h) = ??
- The *probability mass* of words seen once?
  - P^* = 3/25

Once we take 3/25 for unseen words, what will happen to

P(e) = ?

P(f) = ?

P(g) = ?

It will be distributed among unseen words

Good-Turing Smoothing

- In general, take some *probability mass* of n-grams that occurred *t*+1 times and redistribute it to the n-grams that occurred *t* times.
- *P*(words with zero-count) = $\frac{N_1}{N}$
- *P*(words with non-zero-count) = $\frac{(c+1)N_{c+1}}{N_c * N}$

- N_c → number of words with frequency *c*
  - Observation [a:10, b:5, c:5, d:2, e:1, f:1, g:1] ⇒ *N* = 25
  - N_1 = 3 N_2 = 1 N_3 = 0 N_4 = 0 N_5 = 2 …. N_{10} = 1

- *P*(h) = 3/25
- *P*(e):
  - *C* = 1 → $\frac{(c+1)N_{c+1}}{N_c * N} = \frac{(c+1)N_{c+1}}{N_c * N} = \frac{(1+1)N_2}{N_1 * N} = \frac{(1+1)1}{3 * 25} = \frac{2}{75}$

Good-Turing Smoothing

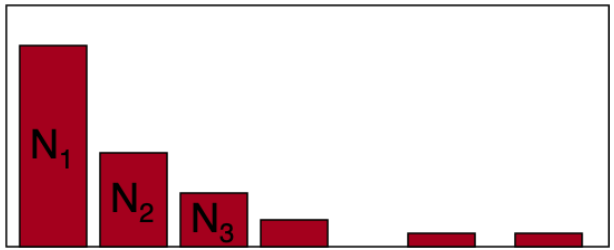
- In general, take some *probability mass* of n-grams that occurred *t*+1 times and redistribute it to the n-grams that occurred *t* times.
- *P*(words with zero-count) = $\frac{N_1}{N}$
- *P*(words with non-zero-count) = $\frac{(c+1)N_{c+1}}{N_c * N}$

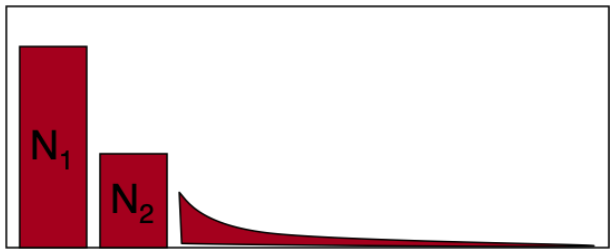
- N_c → number of words with frequency *c*
  - Observation [a:10, b:5, c:5, d:2, e:1, f:1, g:1] ⇒ *N* = 25
  - N_1 = 3 N_2 = 1 N_3 = 0 N_4 = 0 N_5 = 2 …. N_{10} = 1

- *P*(h) = 3/25
- *P*(e):
  - *C* = 1 → $\frac{(c+1)N_{c+1}}{N_c * N} = \frac{(c+1)N_{c+1}}{N_c * N} = \frac{(1+1)N_2}{N_1 * N} = \frac{(1+1)1}{3 * 25} = \frac{2}{75}$

Good-Turing Smoothing

- What if N_{C+1} = 0 for some C?
- Solution: Smooth the counts
  - Simple Good Turing (SGT) [Gale and Sampson, 1995] (Linear regression in log space)





https://grsampson.net/AGtf1.html

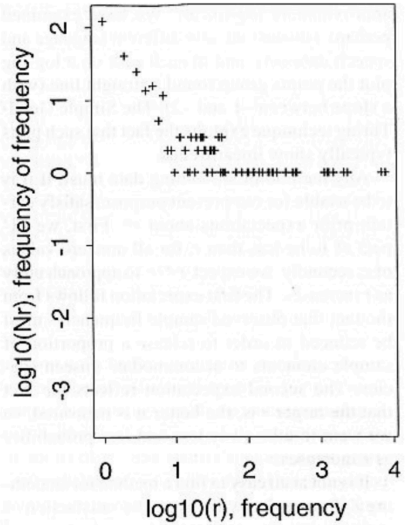
Simple Good-Turing Smoothing

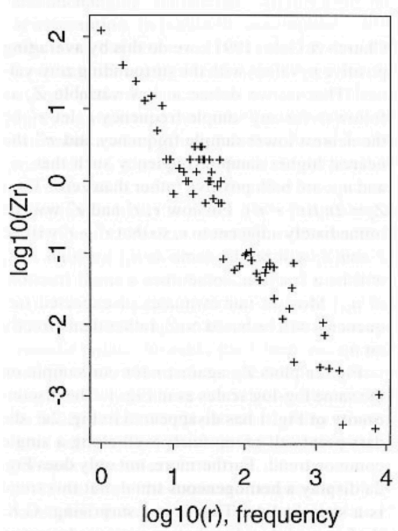
- Linear regression in log space.
  - Find a line that best fit to the $(\log Z_r, \log r)$ point.

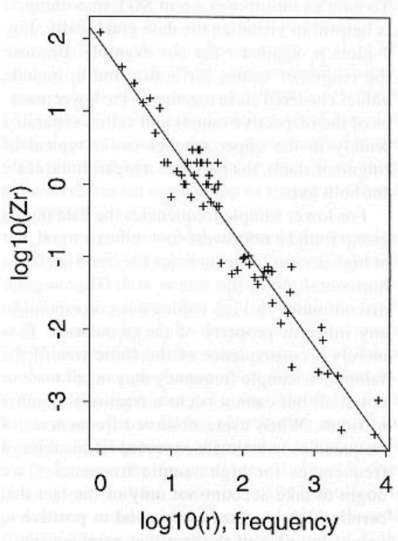
$$Z_r = \frac{2N_r}{r'' - r'}$$

where r'' = nearest higher sample freq

r' = nearest lower sample freq







$*r = c$

X

https://grsampson.net/AGtf1.html

Simple Good-Turing Smoothing

- *P*(words with zero-count) $= \frac{N_1}{N}$
- *P*(words with non-zero-count) $= \frac{(r+1)S(N_{r+1})}{S(N_r) * N}$

Smooth values

Observation [a:10, b:5, c:5, d:2, e:1, f:1, g:1]

$N_1 = 3$ $N_2 = 1$ $N_3 = 0$ $N_4 = 0$ $N_5 = 2$ …. $N_{10} = 1$

$$Z_r = \frac{2N_r}{r'' - r'} \qquad Z_2 = \frac{2N_2}{5-1} = \frac{2}{4} = 0.5$$

where $r'' = 5$ and $r' = 1$

X

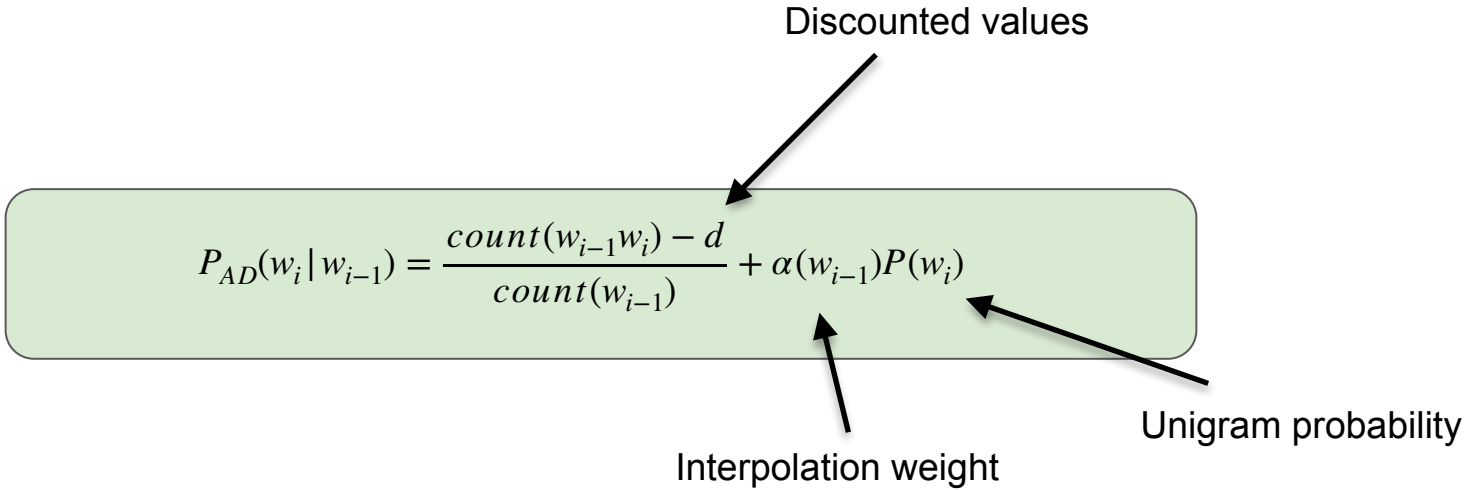
Good-Turing Smoothing

- Reconstructed counts for Good-Turing smoothing
- Corpus
  - AP Newswire
  - N = 22M

Average discount (*d*) ~ 0.75

Count c	Good Turing c*
0	.0000270
1	0.446
2	1.26
3	2.24
4	3.24
5	4.22
6	5.19
7	6.21
8	7.24
9	8.25

Absolute discounted



Kneser-Ney smoothing

- Kneser-Ney smoothing targets the unigram probability $P(w_i)$
- Intuition,
  - Some word can have a high unigram probability; however, it always occur in some specific context.
  - Assign such a word a lower probability

- E.g., I can't read without my reading _____ *glasses* *fransisco*
  - In general, *fransisco* is more common than *glasses*, the unigram model will prefer *francisco* over *glasses.*
  - However, *fransisco* usually occur after *San.*
    - Therefore, the model should assign lower probability to it.

- Continuation probability
  - How likely is w_i to appear as the novel continuation
  - Replace $P(w_i)$ with $P_{Continuation}(w_i)$

Kneser-Ney smoothing

$$P_{AD}(w_i \mid w_{i-1}) = \frac{count(w_{i-1}w_i) - d}{count(w_{i-1})} + \alpha(w_{i-1}) P_{Continuation}(w_i)$$

$$P_{Continuation}(w_i) = \frac{|\{w_{i-1} : count(w_{i-1}w_i) > 0\}|}{|\{(w_{j-1}w_j) : count(w_{j-1}w_j) > 0\}|)}$$

The number of word types that can follow w_{i-1}

$$\alpha(w_{i-1}) = \frac{d}{count(w_{i-1})} |\{w : count(w_{i-1}w) > 0\}|$$

where $(w_{j-1}w_j)$ is all possible bigrams.

$$P_{AD}(w_i \mid w_{i-1}) = \frac{max(count(w_{i-1}w_i) - d, 0)}{count(w_{i-1})} + \alpha(w_{i-1}) P_{Continuation}(w_i)$$

Thanks